

UNIVERSIDAD NACIONAL DE CÓRDOBA

FACULTAD DE CIENCIAS EXACTAS, FÍSICAS Y NATURALES
CÁTEDRA DE PROGRAMACIÓN CONCURRENTE Y PARALELA

TRABAJO PRÁCTICO INTEGRADOR

“Sistema de procesamiento de transacciones de pago”

Grupo: Hardcodeados

Estudiantes:

Fumero, Antonio Esequiel

Arias Paz, Andres Natanael

Moyano Boscatto, Matías

Córdoba, Ulises Baltasar

Profesores:

Ing. Ventre, Luis Orlando

Ing. Ludemann, Mauricio

Ing. Carranza, Agustín

Noviembre / 2026

CONTENIDOS

1. Objetivo

Implementar en Java un programa concurrente basado en una red de Petri que modela un sistema de procesamiento de transacciones de pago (PSP) haciendo uso de un monitor de concurrencia.

Dicho sistema recibe transacciones desde una cola de arribo, las admite, las enruta hacia uno de tres flujos de procesamiento según su tipo, y finalmente las deposita en un buffer de salida para su liquidación.

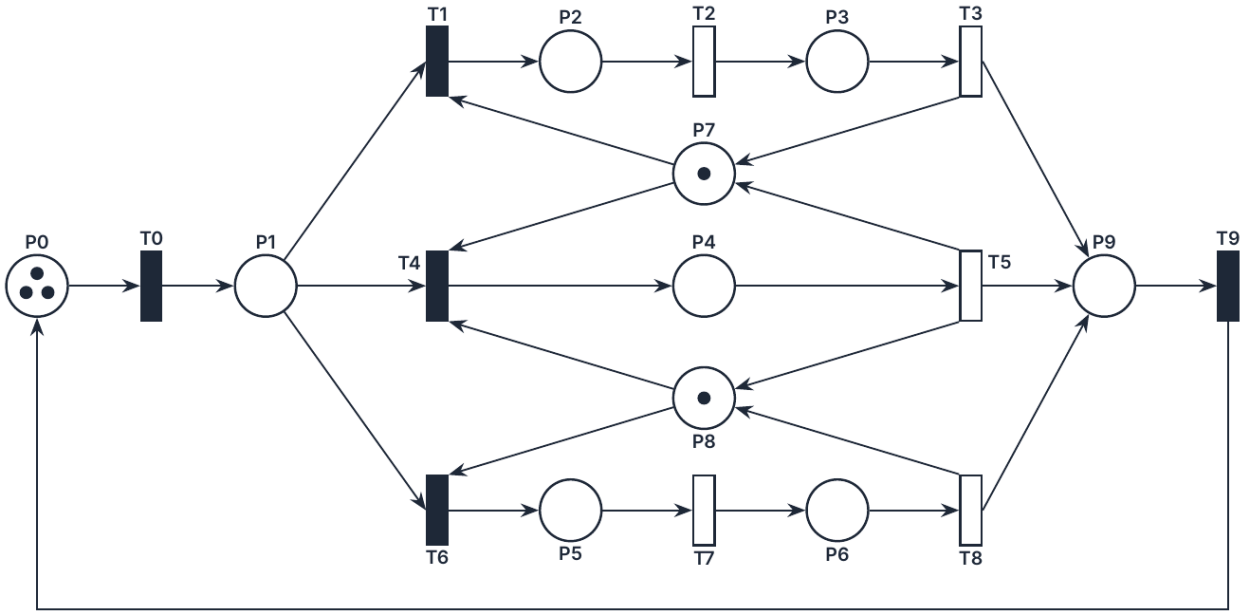


Figura 1: red del enunciado. (Negras: transiciones inmediatas; blancas: temporales. Todos los arcos pesan 1.)

2. Análisis de la red

La red tiene 10 plazas y 10 transiciones, es ordinaria (todos los arcos pesan 1) y pura (ninguna plaza es entrada y salida de la misma transición). Su único conflicto está en P1, donde T1, T4 y T6 compiten por cada transacción admitida.

Marcado inicial: M0 = [3, 0, 0, 0, 0, 0, 0, 1, 1, 0]. Las transiciones temporales son T2, T3, T5, T7 y T8; el resto son inmediatas.

Solo tres transacciones circulan por la red: las 3 marcas iniciales de P0. Cada una se admite (T0), se rutea a uno de tres flujos:

- Pago con tarjeta de crédito/débito: iniciado al dispararse T1;
- Pago de alto riesgo: mediante el disparo de T4;
- Transferencia bancaria: con el disparo de T6;

y termina en el buffer de salida P9, desde donde T9 la devuelve a la cola de arribo. Los flujos compiten por dos recursos: la sesión con el gateway de tarjetas (P7) y el slot del motor antifraude (P8).

Decisiones principales

Tema	Decisión	Sección
Hilos	9 hilos en 5 segmentos (3 + 1 + 1 + 1 + 3), según el algoritmo del artículo de referencia	5
Monitor	Mutex de entrada, una cola de condición por transición y paso del mutex al hilo despertado, como en el diagrama de la cátedra	6
Políticas	La política decide el flujo de cada transacción en el conflicto de P1: aleatoria uniforme, o priorizada 8:1:1 a favor de alto riesgo	8
Tiempo	Semántica de tiempo de sensibilización: el hilo espera su ventana fuera del monitor	9
Corte	T0 se dispara exactamente 200 veces; la corrida terminó con el disparo 200 de T9 y sin hilos activos	10

2.1 Tabla de estados

Plaza	Tipo	Estado del sistema
P0	Idle	Cola de arribo: transacciones esperando ingresar
P1	Acción	Transacción admitida (parsing, schema y autenticación superados), lista para rutear
P2	Acción	Tarjeta: autorización solicitada al issuer, esperando respuesta
P3	Acción	Tarjeta: autorización aprobada, captura de fondos en curso
P4	Acción	Alto riesgo: scoring antifraude con consulta en tiempo real al gateway
P5	Acción	Transferencia: validación de la cuenta destino
P6	Acción	Transferencia: ejecución en la red bancaria
P7	Recurso	Sesión con el gateway de tarjetas libre
P8	Recurso	Slot del motor antifraude libre
P9	Acción	Buffer de salida: lista para confirmar al cliente y liquidar

2.2 Tabla de eventos

Transición	Tipo	Evento
T0	Inmediata	Ingreso y admisión de una transacción
T1	Inmediata	Ruteo a tarjeta: toma la sesión del gateway y pide la autorización
T2	Temporal	Llega la respuesta del issuer
T3	Temporal	Captura confirmada: libera el gateway y deja la transacción en el buffer
T4	Inmediata	Ruteo a alto riesgo: toma el gateway y el antifraude a la vez
T5	Temporal	Fin del scoring: libera ambos recursos y deja la transacción en el buffer
T6	Inmediata	Ruteo a transferencia: toma el slot antifraude
T7	Temporal	Cuenta destino validada
T8	Temporal	Transferencia ejecutada: libera el antifraude y deja la transacción en el buffer
T9	Inmediata	Confirmación al cliente y envío a liquidación; la transacción vuelve a la cola de arribo

2.3 Matriz de incidencia

I = W+ − W−, con filas = plazas y columnas = transiciones. Como la red es pura, W− son los −1 de I y W+ son los +1.

	T0	T1	T2	T3	T4	T5	T6	T7	T8	T9
P0	−1	0	0	0	0	0	0	0	0	+1
P1	+1	−1	0	0	−1	0	−1	0	0	0
P2	0	+1	−1	0	0	0	0	0	0	0
P3	0	0	+1	−1	0	0	0	0	0	0
P4	0	0	0	0	+1	−1	0	0	0	0
P5	0	0	0	0	0	0	+1	−1	0	0
P6	0	0	0	0	0	0	0	+1	−1	0
P7	0	−1	0	+1	−1	+1	0	0	0	0
P8	0	0	0	0	−1	+1	−1	0	+1	0

	T0	T1	T2	T3	T4	T5	T6	T7	T8	T9
P9	0	0	0	+1	0	+1	0	0	+1	−1

El código dispara con la ecuación de estado. Una transición Tj está sensibilizada si el marcado alcanza para su columna de W−, y su disparo suma la columna j de I:

$$M_{k+1} = M_k + I \cdot \sigma_j \qquad T_j \text{ sensibilizada} \iff M_k \geq W^- \cdot \sigma_j$$

2.4 Conflictos

Hay tres conflictos estructurales: P1 ⟶ {T1, T4, T6} (ruteo), P7 ⟶ {T1, T4} y P8 ⟶ {T4, T6} (recursos). Las tres transiciones forman un único conjunto en conflicto, {T1, T4, T6}: decidir cuál toma la transacción de P1 decide también quién toma los recursos. Las transiciones temporales no tienen conflicto, porque cada una tiene una única plaza de entrada propia.

3. Propiedades de la red

La red no tiene deadlock, es viva, reversible y 3-acotada, pero no es segura: P0, P1 y P9 llegan a tener 3 marcas.

Las propiedades se determinaron con PIPE (módulos *State Space Analysis*, *Invariant Analysis* y *Reachability/Coverability Graph*). Además, las verificamos con un cálculo propio del grafo de alcanzabilidad (`herramientas/analisis_red.py`): 52 marcados alcanzables y 134 arcos.

Propiedad	Resultado	Justificación
Deadlock	No tiene	Ninguno de los 52 marcados alcanzables deja la red sin transiciones sensibilizadas
Acotamiento	3-acotada	P0, P1 y P9 llegan a 3 marcas; P2 a P8, como máximo a 1 (lo garantizan los P-invariantes)
Seguridad	No es segura	P0, P1 y P9 pueden tener más de una marca
Vivacidad	Viva	El grafo es fuertemente conexo y aparecen las 10 transiciones: desde cualquier marcado se puede volver a disparar cualquier transición
Reversibilidad	Reversible	Desde cualquier marcado alcanzable se puede volver a M0

Qué significa para el programa. Al ser acotada, ninguna plaza crece sin límite: los "buffers" P1 y P9 nunca superan 3 transacciones. Que no sea segura significa que hasta 3 hilos pueden trabajar a la vez con marcas de una misma plaza, y eso justifica 3 hilos en T0 y en T9 (sección 5). Sin deadlock y viva, el programa nunca se traba y cualquier flujo se puede volver a ejecutar. Por ser reversible, al limitar T0 a 200 disparos la red vuelve a M0 al final de cada corrida, y lo verificamos en todas.

Capturas de PIPE (agregar al informe):

- ☐ *State Space Analysis*: bounded = true, safe = false, deadlock = false
- ☐ *Invariant Analysis*: P-invariantes, T-invariantes y ecuaciones de la sección 4
- ☐ *Reachability Graph*: 52 estados
- ☐ *Incidence & Marking*: matriz de la sección 2.3

4. Invariantes de plaza y de transición

La red tiene 3 T-invariantes, uno por flujo de pago, y 3 P-invariantes: uno conserva las 3 transacciones y los otros dos garantizan la exclusión mutua de cada recurso compartido.

4.1 T-invariantes ($I \cdot x = 0$)

Invariante	Transiciones	Significado
IT1	T0 T1 T2 T3 T9	Pago con tarjeta: admisión, autorización, captura y salida
IT2	T0 T4 T5 T9	Pago de alto riesgo: admisión, scoring con gateway y antifraude, salida
IT3	T0 T6 T7 T8 T9	Transferencia: admisión, validación de la cuenta, ejecución y salida

Cada T-invariante es el ciclo completo de una transacción: disparar esa secuencia devuelve la red al mismo marcado. Los tres comparten T0, antes del conflicto en P1, y T9, después del join en P9. Esos dos puntos definen los segmentos de hilos de la sección 5.

4.2 P-invariantes ($x^T \cdot I = 0$)

Invariante	Ecuación	Significado
PI1	M(P0) + M(P1) + M(P2) + M(P3) + M(P4) + M(P5) + M(P6) + M(P9) = 3	Siempre hay exactamente 3 transacciones en el sistema: no se crean ni se pierden
PI2	M(P2) + M(P3) + M(P4) + M(P7) = 1	La sesión del gateway está libre o la usa exactamente un flujo: tarjeta o alto riesgo
PI3	M(P4) + M(P5) + M(P6) + M(P8) = 1	El slot antifraude está libre o lo usa exactamente un flujo: alto riesgo o transferencia

PI2 y PI3 prueban la exclusión mutua sobre cada recurso. De ellos se deduce algo clave para el análisis de tiempos: alto riesgo nunca se ejecuta junto con otro flujo, mientras que tarjeta y transferencia sí pueden ejecutarse en paralelo, porque no comparten recurso.

Las constantes salen del marcado inicial ($x^T \cdot M0$), y el programa verifica las tres ecuaciones después de cada disparo (sección 10).

5. Cantidad de hilos y responsabilidades

El sistema necesita 9 hilos para el mayor paralelismo posible: 3 para T0, uno por cada flujo de pago y 3 para T9.

5.1 Algoritmo aplicado

Seguimos el algoritmo del artículo de referencia (Ventre y Micolini, CACIC 2021), que es el que formalizan los casos 1 y 2 del enunciado:

1. Se toman los T-invariantes de la sección 4.
2. De las plazas de cada invariante se quitan la idle (P0) y las de recursos (P7 y P8). Quedan las plazas de acción: P1, P2, P3, P4, P5, P6 y P9.
3. Se parten los invariantes en segmentos. El conflicto en P1 es el Caso 1: un segmento con la transición anterior al conflicto (T0) y uno por invariante. El join en P9 es el Caso 2: un segmento con las transiciones restantes (T9).
4. Cada segmento lleva tantos hilos como marcas simultáneas pueden tener sus plazas de acción, mirando los 52 marcados alcanzables.

Segmento	Transiciones	Plazas de acción	Caso	Máx. marcas	Hilos
SA	T0	P1	Antes del conflicto (1)	3	3
SB	T1 T2 T3	P2 P3	Rama tarjeta (1)	1, por PI2	1
SC	T4 T5	P4	Rama alto riesgo (1)	1, por PI2 y PI3	1
SD	T6 T7 T8	P5 P6	Rama transferencia (1)	1, por PI3	1
SE	T9	P9	Después del join (2)	3	3
Total					**9**

Como hay 3 transacciones (PI1), nunca hay más de 3 hilos con trabajo útil a la vez; el resto espera en las colas del monitor.

Sobre el Caso 1. El texto del enunciado habla de "un hilo encargado de la/s transición/es anterior/es al conflicto". El artículo y el ejemplo de la Figura 2 cuentan las marcas simultáneas: allí P1 tiene 2 marcas y T1 lleva 2 hilos. Por eso SA tiene 3 hilos. La cantidad por segmento es un dato de `ConfiguracionRed`, así que cambiarla no toca el monitor.

5.2 Responsabilidad de cada hilo

Cada hilo repite la secuencia de su segmento: SA dispara T0; SB, T1 → T2 → T3; SC, T4 → T5; SD, T6 → T7 → T8; SE, T9. El gráfico de responsabilidades por colores está en la figura de esta sección.

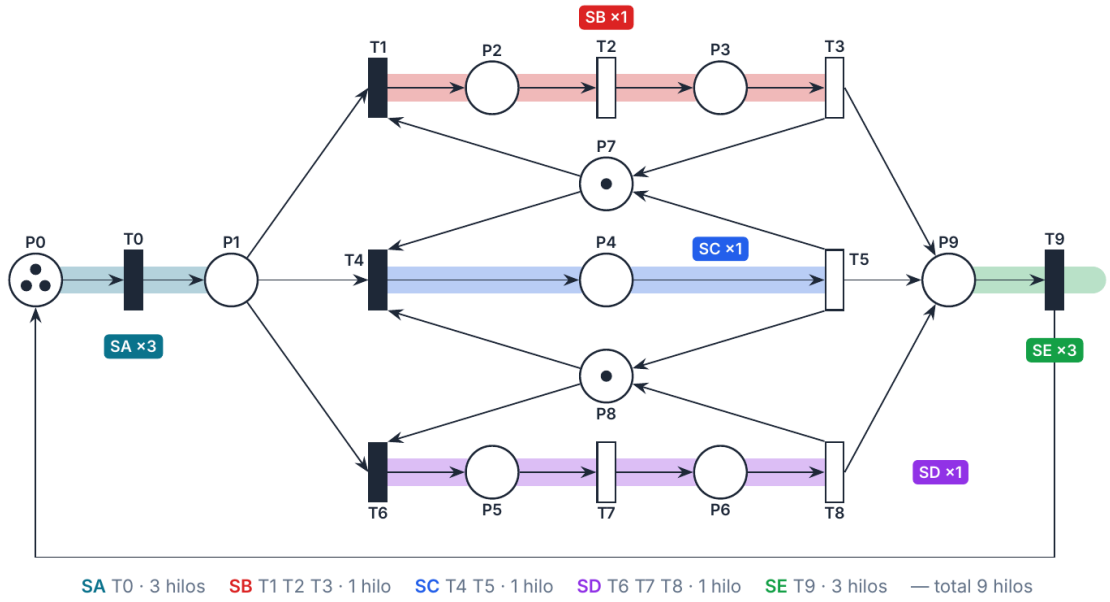


Figura 2. Responsabilidades de los hilos: cada color es un segmento y el número indica cuántos hilos lo ejecutan (en total, 9).

6. Diseño e implementación

El proyecto separa la lógica de la red (`RedDePetri`), la política (`Politica`) y las acciones (los hilos `Disparador`). El `Monitor` solo coordina y no conoce la red: todo lo propio de esta red está en `ConfiguracionRed`.

6.1 Clases

Paquete	Clase	Responsabilidad
(raíz)	`Main`	Inicia una corrida: `java -cp out Main [aleatoria|priorizada] [etiqueta] [perfil]`
(raíz)	`Experimentos`	Corre el plan de experimentos y deja `logs/resultados.csv`
simulacion	`Simulacion`	Arma red, política, monitor y los 9 hilos; corta en 200 invariantes y termina los hilos
simulacion	`Resultado`	Datos de una corrida: duración, invariantes por tipo y verificaciones
monitor	`MonitorInterface`	Interfaz del enunciado: `boolean fireTransition(int transition)`

Paquete	Clase	Responsabilidad
monitor	`Monitor`	Mutex, ciclo de disparo y paso del mutex; su único método público es `fireTransition`
monitor	`Colas`	Una cola de condición por transición: semáforo más contador de hilos esperando
monitor.politicas	`Politica`	Interfaz: `permite`, `cual` y `notificarDisparo`
monitor.politicas	`PoliticaPorConflictos`	Turno por conjunto de transiciones en conflicto
monitor.politicas	`PoliticaAleatoria`, `PoliticaPriorizada`	Cómo se elige el próximo turno: al azar o con pesos 1:8:1
red	`RedDePetri`	Marcado, sensibilizado y disparo con la ecuación de estado; verifica P-invariantes; calcula los conflictos desde W−
red	`ControlTemporal`	Ventanas [α, β], timestamps de sensibilización y banderas de espera
red	`ConfiguracionRed`	Única clase con datos de esta red: W−, W+, M0, invariantes, regex, tiempos, pesos y segmentos
red	`Segmento`	Secuencia de transiciones y cantidad de hilos de un segmento
hilos	`Disparador`	`Runnable` que repite la secuencia de su segmento
hilos	`FabricaHilos`	`ThreadFactory` que registra los hilos creados para verificar que ninguno quede vivo
registro	`Log`	Secuencia de disparos y tabla de eventos, escritas con el mutex tomado
registro	`AnalizadorInvariantes`	Expresión regular de T-invariantes sobre el log

El diagrama de clases completo está en `docs/diagrama_clases.png` y al final de esta sección.

6.2 Algoritmo de fireTransition

Sigue los diagramas de secuencia de la cátedra (monitor sin tiempo y con tiempo). En cada vuelta del ciclo el hilo tiene el mutex:

1. Si la transición no está sensibilizada, la política no la permite u otro hilo ya espera su ventana, el hilo se anota en la cola de esa transición, libera el mutex y se bloquea.
2. Si está sensibilizada pero su ventana todavía no se abrió, libera el mutex, duerme lo que falta fuera del monitor, vuelve a tomar el mutex y reintenta.

3. Si puede, dispara: ecuación de estado, verificación de P-invariantes, aviso a la política y registro en el log.
4. Calcula despertables = sensibilizadas ∧ esperando ∧ permitidas. Si hay alguna, la política elige cuál, ese hilo se despierta y recibe el mutex. Si no hay ninguna, libera el mutex.

```
public boolean fireTransition(int transition) {
    if (!tomarMutex()) return false;
    while (true) {                                  // en cada vuelta el hilo tiene
el mutex
        if (!puedeIntentar(transition)) {           // no sensibilizada, sin
permiso o alguien espera su ventana
            if (!esperarEnCola(transition)) return false;
            continue;                               // lo despertaron: ya tiene el
mutex
        }
        long ahora = System.nanoTime();
        long esperaMs = tiempo.milisHastaVentana(transition, ahora);
        if (esperaMs > 0) {                         // antes de la ventana
            if (!dormirFueraDelMonitor(transition, esperaMs)) return false;
            continue;
        }
        if (tiempo.ventanaVencida(transition, ahora)) { mutex.release(); return
false; }
        disparar(transition, ahora);                // ecuación de estado +
P-invariantes + log
        despertarOSalir();                          // pasa el mutex o lo libera
        return true;
    }
}
```

6.3 Por qué es correcto

- **Exclusión mutua.** Un `Semaphore(1, true)` protege la red, las colas, la política, el control temporal y el log: ninguno se toca sin el mutex. Que sea justo (FIFO) evita la inanición en la entrada.
- **Paso del mutex.** Quien despierta a otro hilo sale sin liberar el mutex, y el despertado continúa con el control. Así ningún hilo nuevo le roba la transición que estaba sensibilizada para él (idea de *signal and exit*).
- **Sin despertares perdidos.** El contador de la cola sube antes de soltar el mutex, y el semáforo guarda el permiso aunque el hilo todavía no haya llegado a `acquire()`.
- **Acciones fuera del monitor.** El monitor solo decide si el hilo avanza; las esperas temporales también se hacen sin el mutex.
- **Monitor agnóstico.** `Monitor` no contiene números de transición. Lo que necesita lo pregunta a `RedDePetri`, `ControlTemporal` y `Politica`, que se construyen con los datos de `ConfiguracionRed`. Para ejecutar otra red solo cambia esa clase.
- **Objetos y colecciones.** Se usan `List`/`ArrayList`, `Map`/`LinkedHashMap`, arreglos, `Semaphore`, `AtomicInteger`, `CountDownLatch`, `ExecutorService` y `ThreadFactory`.

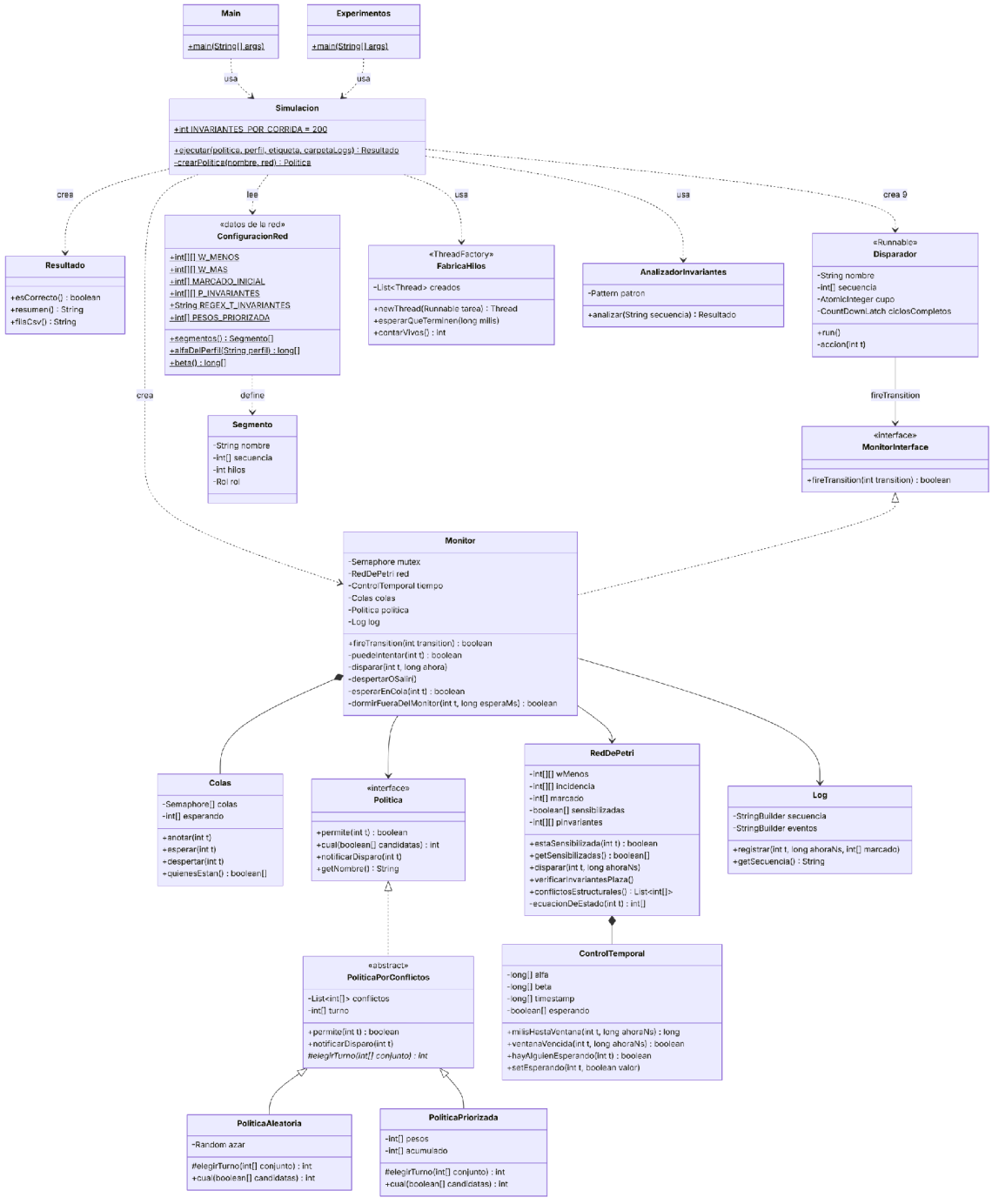


Figura 3. Diagrama de clases (`docs/diagrama_clases.png`).

7. Diagrama de secuencia de un disparo

El diagrama sigue una llamada a `fireTransition(t)` de principio a fin y muestra los tres momentos en que interviene la política. Está en `docs/diagrama_secuencia.png` y al final de esta sección.

- **Participantes:** el hilo (`Disparador`), `Monitor`, el mutex, `RedDePetri`, `ControlTemporal`, `Politica`, `Colas` y `Log`.
- **Fragmento loop:** el hilo repite el intento hasta disparar; en cada vuelta tiene el mutex.
- **Fragmento alt, cuatro ramas:** (1) no puede intentar y se bloquea en la cola de t; (2) está antes de la ventana y duerme fuera del monitor; (3) se venció la ventana y devuelve `false`; (4) dispara.
- **Uso de la política:** `permite(t)` antes de intentar (el turno del conflicto), `notificarDisparo(t)` después del disparo (elige el próximo turno) y `permite(i)` + `cual(despertables)` para elegir a quién despertar.
- **Fragmento alt final:** si hay un hilo elegido, el mutex pasa a ese hilo sin liberarse; si no, se libera.

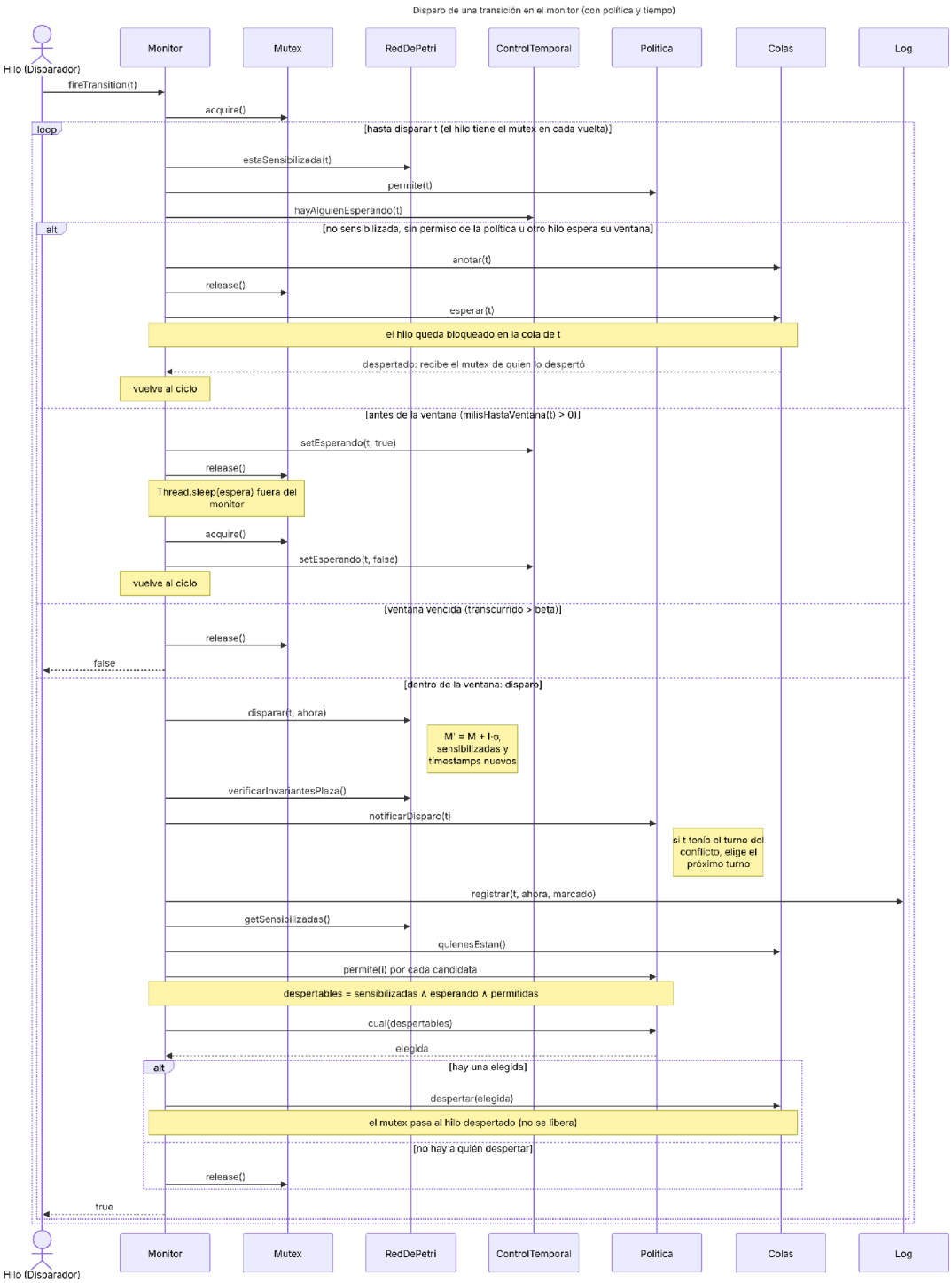


Figura 4. Diagrama de secuencia de un disparo (`docs/diagrama_secuencia.png`).

8. Políticas

Las dos políticas deciden por qué flujo sale cada transacción en el conflicto de P1. La aleatoria sortea el flujo con probabilidad 1/3; la priorizada envía 8 de cada 10 transacciones a alto riesgo.

8.1 Dónde actúa la política

- `RedDePetri.conflictosEstructurales()` calcula desde W^- los conjuntos de transiciones que comparten plazas de entrada. En esta red hay uno solo: {T1, T4, T6}.
- Para ese conjunto la política mantiene un **turno**: solo la transición con el turno puede tomar la transacción de P1. Las otras dos esperan en su cola aunque estén sensibilizadas. Cuando la transición con turno se dispara, la política elige el próximo.
- Además, la política elige a quién despertar entre los despertables (`cual`), como en el diagrama de la cátedra.

8.2 Por qué no alcanza con elegir a quién despertar

Alto riesgo necesita los dos recursos a la vez; tarjeta y transferencia, uno cada una. Si la política solo eligiera a quién despertar, el hilo de tarjeta volvería a tomar P7 apenas lo libera: entra con T1 sensibilizada y nadie consulta a la política. Lo mismo pasa con transferencia y P8, así que alto riesgo solo corre cuando ambos recursos quedan libres por casualidad.

Lo comprobamos con una versión de comparación (`herramientas/comparacion/SoloDespertar.java`, 3 corridas por variante, tiempos base):

Variante	Tarjeta	Alto riesgo	Transferencia
Solo elige a quién despertar, al azar	77 a 89	22 a 47	76 a 89
Solo elige a quién despertar, con prioridad a T4	76 a 86	28 a 48	76 a 86

Dar prioridad a T4 al despertar no cambia el resultado: alto riesgo queda entre 11 % y 24 % con cualquiera de las dos. Por eso nuestra política decide en el conflicto mismo.

8.3 Política aleatoria

- **Turno:** se sortea con probabilidad uniforme entre T1, T4 y T6 (`java.util.Random`).
- **A quién despertar:** al azar entre las candidatas.
- **Esperado:** cada flujo sale con probabilidad 1/3, es decir 66,7 ± 6,7 transacciones de 200 (desvío de una binomial: $\sqrt{(200 \cdot 1/3 \cdot 2/3)} \approx 6{,}7$).

8.4 Política priorizada

- **Turno:** round-robin ponderado suave con pesos T1 = 1, T4 = 8 y T6 = 1 (`ConfiguracionRed.PESOS_PRIORIZADA`). De cada 10 turnos, 8 son de alto riesgo y quedan intercalados: H H H C H H T H H H.
- **Esperado:** con 200 transacciones da exactamente 160 de alto riesgo, 20 de tarjeta y 20 de transferencia, en todas las corridas.
- **A quién despertar:** la candidata de mayor peso.
- **Por qué no prioridad estricta:** alto riesgo siempre tiene una transacción esperando, porque hay 3 en el sistema. Con pesos 0:1:0, T1 y T6 no se dispararían nunca (inanición). Los pesos 1:8:1 priorizan sin dejar sin servicio a los otros dos flujos.

9. Semántica temporal

Las cinco transiciones temporales usan semántica de tiempo de sensibilización: cada una puede dispararse α ms después de quedar sensibilizada, y el hilo espera esa ventana fuera del monitor.

9.1 Implementación

- **Intervalo [α, β] por transición** (`ControlTemporal`). Las inmediatas tienen α = 0. Todas tienen β = ∞ (`SIN_LIMITE`): con semántica débil, un β finito podría dejar una transacción trabada si su hilo llega tarde. La rama "ventana vencida" está implementada, pero con β = ∞ no ocurre.
- **Timestamp de sensibilización.** Después de cada disparo, `RedDePetri.disparar` le pasa a `ControlTemporal` las sensibilizadas de antes y de después. Las recién sensibilizadas (y la disparada, si sigue sensibilizada) guardan `System.nanoTime()`; las que dejaron de estarlo pierden el timestamp.
- **Antes de la ventana.** Si transcurrido < α, el hilo marca `esperando`, libera el mutex, duerme lo que falta (redondeado hacia arriba a milisegundos), vuelve a tomar el mutex y reintenta. Si mientras duerme otro hilo pide la misma transición, ese hilo va a la cola, como en el diagrama 10.2.2 de la cátedra.
- **Por qué fuera del monitor.** Si el hilo durmiera con el mutex, toda la red se frenaría durante α: tarjeta y transferencia no podrían avanzar en paralelo.

9.2 Tiempos elegidos (perfil base)

Transición	Etapa	α (ms)
T2	Respuesta del issuer	120
T3	Captura de fondos	80
T5	Scoring antifraude con consulta al gateway	140
T7	Validación de la cuenta destino	70
T8	Ejecución en la red bancaria	130

Por flujo, tarjeta ocupa el gateway 200 ms (T2 + T3), alto riesgo ocupa los dos recursos 140 ms y transferencia ocupa el antifraude 200 ms. Elegimos estos valores para que las dos políticas completen 200 invariantes en unos 30 s, dentro del rango de 20 a 40 s (sección 12).

Para el análisis hay otros perfiles en `ConfiguracionRed`: `riesgo_lento` (T5 = 280), `tarjeta_lenta` (T2 = 240, T3 = 160), `rapido` (todos a la mitad) y `sin_tiempo` (todos en 0), que es el monitor sin semántica temporal de la primera etapa.

10. Verificación

Las 50 corridas del plan de experimentos pasaron las tres verificaciones: 44.342 chequeos de P-invariantes sin fallas, residuo vacío en la expresión regular y ningún hilo activo al terminar.

10.1 P-invariantes después de cada disparo (requerimiento 12)

- Después de cada disparo, todavía con el mutex tomado, el monitor llama a `RedDePetri.verificarInvariantesPlaza()`. Para cada P-invariante compara x · M con x · M0; si no coinciden, lanza una excepción con el marcado.
- Al construirse, `RedDePetri` también comprueba que cada vector de la configuración sea de verdad un P-invariante ($x^T \cdot I = 0$).
- Hay un chequeo por disparo: 840 por corrida con la priorizada y entre 918 y 942 con la aleatoria. En total fueron 44.342 chequeos, ninguno falló.

10.2 T-invariantes con expresión regular (requerimiento 13)

El monitor escribe cada disparo en el log con el mutex tomado, así que la secuencia de `logs/<política>_<perfil>_<corrida>_disparos.txt` respeta el orden real. Como los tres invariantes comparten T0 y T9, se usa una sola expresión con todos los caminos (`ConfiguracionRed.REGEX_T_INVARIANTES`):

```
T0(.*?)(?:T1(?<it1>.*?)T2(.*?)T3(.*?)|T4(?<it2>.*?)T5(.*?)|T6(?<it3>.*?)T7(.*?)T8(.*?))T9
```

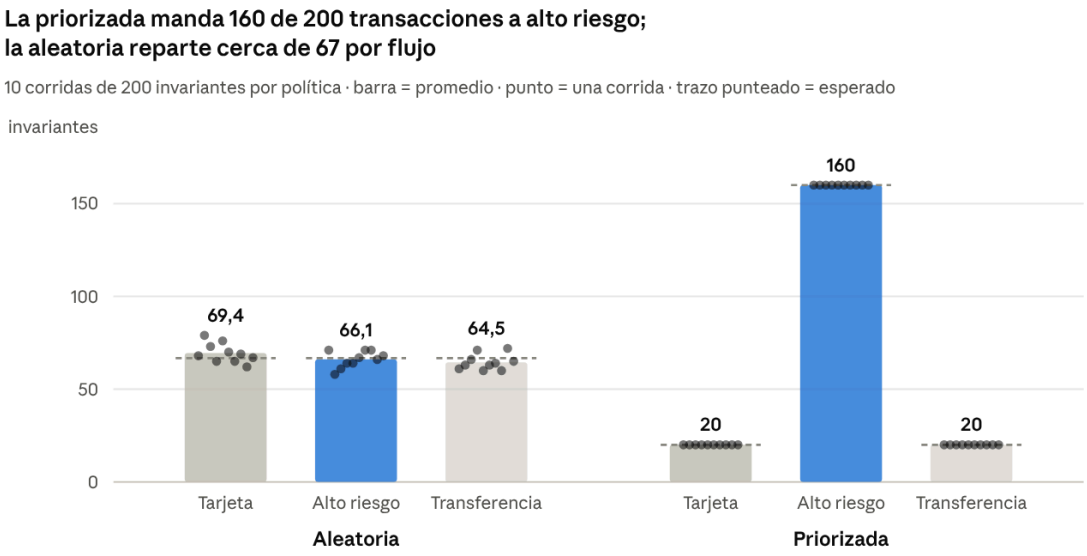
- `AnalizadorInvariantes` reemplaza cada coincidencia por la concatenación de sus grupos `(.*?)`: borra las transiciones del invariante encontrado y conserva lo que estaba intercalado. Repite hasta que no hay más coincidencias.
- El grupo con nombre que participó (`it1`, `it2` o `it3`) dice de qué tipo era el invariante.
- Como T0 se dispara exactamente 200 veces y la red vuelve a M0, el residuo tiene que quedar vacío y las cuentas tienen que sumar 200. Así fue en las 50 corridas, siempre en 3 pasadas.
- Comprobamos que detecta errores: si a un log se le quita un T5, quedan 199 invariantes y el residuo `T9T0T4`; una secuencia con el orden cambiado (`T0T2T1T3T9`) no coincide.

10.3 Finalización sin hilos activos (requerimiento 3)

1. Los 3 hilos de SA comparten un cupo de 200 (`AtomicInteger`): reservan cada T0 antes de dispararla, así que entran exactamente 200 transacciones.
2. Los 3 hilos de SE descuentan un `CountDownLatch(200)` después de cada T9.
3. Cuando el latch llega a 0, `Simulacion` llama a `shutdownNow()` del `ExecutorService`. Los hilos bloqueados en las colas reciben la interrupción, `fireTransition` devuelve `false` sin tener el mutex y cada `Disparador` termina.
4. Se espera a cada hilo creado por `FabricaHilos` y se cuentan los vivos: 0 en las 50 corridas, con marcado final igual a M0.

11. Resultados de las corridas

La política priorizada manda exactamente 160 de cada 200 transacciones a alto riesgo; la aleatoria reparte en promedio 69,4, 66,1 y 64,5 por flujo, cerca del 66,7 esperado.



logs/resultados.csv · corridas E1 (aleatoria) y E2 (priorizada), tiempos base

- **Aleatoria (E1, 10 corridas):** cada corrida tiene entre 58 y 79 invariantes de cada tipo, dentro de unos 2 desvíos del esperado (66,7 ± 6,7). Juntando las 25 corridas aleatorias del plan (5.000 transacciones), el reparto es 1.681, 1.658 y 1.661: la prueba χ^2 no rechaza que sea uniforme (χ^2 = 0,19, p = 0,91).
- **Priorizada (E2, 10 corridas):** 20, 160 y 20 en todas las corridas, igual que en las otras 15 corridas priorizadas del plan. El round-robin ponderado es determinista: 200 turnos con pesos 1:8:1 dan exactamente esa cuenta.
- **Por qué siempre suman 200:** T0 se dispara exactamente 200 veces y la red vuelve a M0, así que cada transacción que entra completa un invariante.
- **Registro (requerimiento 9):** cada corrida deja su resumen en `logs/` y una fila en `logs/resultados.csv`; la tabla de las 50 corridas está en el anexo.

12. Análisis de tiempos

Con los tiempos base, la aleatoria tarda 29,4 s en promedio y la priorizada 30,5 s, dentro del rango pedido de 20 a 40 s. Un modelo analítico simple predice cada corrida con 0,11 s de diferencia promedio.

12.1 Modelo analítico

Cada flujo ocupa sus recursos durante la suma de sus α; las transiciones inmediatas no suman tiempo. Tarjeta ocupa P7 durante tC = α2 + α3 = 200 ms; alto riesgo ocupa P7 y P8 durante tH = α5 = 140 ms; transferencia ocupa P8 durante tT = α7 + α8 = 200 ms.

Como la política asigna los turnos en orden, la transacción k empieza cuando se liberan los recursos de su flujo, y nunca antes que la k − 1. Con s_k el instante en que empieza:

$$s_k = \max\left(s_{k-1},\ \text{libre}(\text{recursos del flujo } k)\right)$$

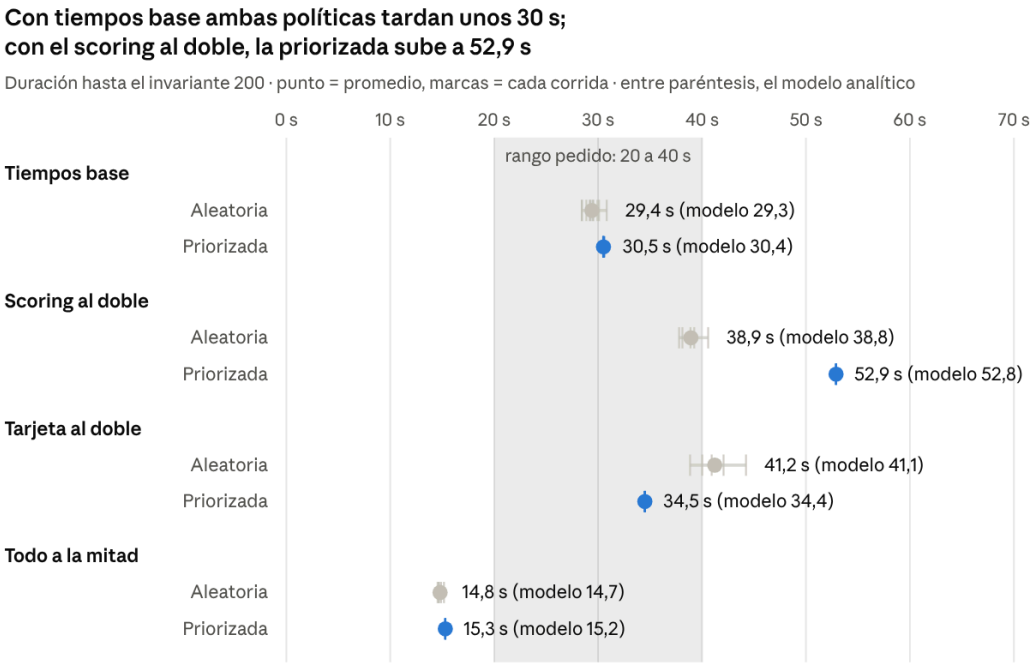
$$T_{total} = \max\left(\text{libre}(P7),\ \text{libre}(P8)\right)$$

Si N_C, N_H y N_T son las cantidades de cada flujo, el total queda entre dos cotas: tarjeta y transferencia solapadas por completo, o todo en serie.

$$N_H t_H + \max(N_C t_C,\ N_T t_T) \ \le\ T_{total} \ \le\ N_H t_H + N_C t_C + N_T t_T$$

- **Priorizada:** con el patrón H H H C H H T H H H, tarjeta y transferencia nunca quedan seguidas y no se solapan. El total es 160 × 140 + 20 × 200 + 20 × 200 ms = 30,4 s, justo la cota superior.
- **Aleatoria:** con unas 67 transacciones por flujo, las cotas son 23,3 y 36,7 s. Tarjeta y transferencia solo se solapan cuando sus turnos quedan seguidos. Simulando el modelo con 20.000 secuencias al azar, el valor esperado es 29,4 s (el 90 % entre 28,2 y 30,5 s).

12.2 Mediciones y variación de tiempos

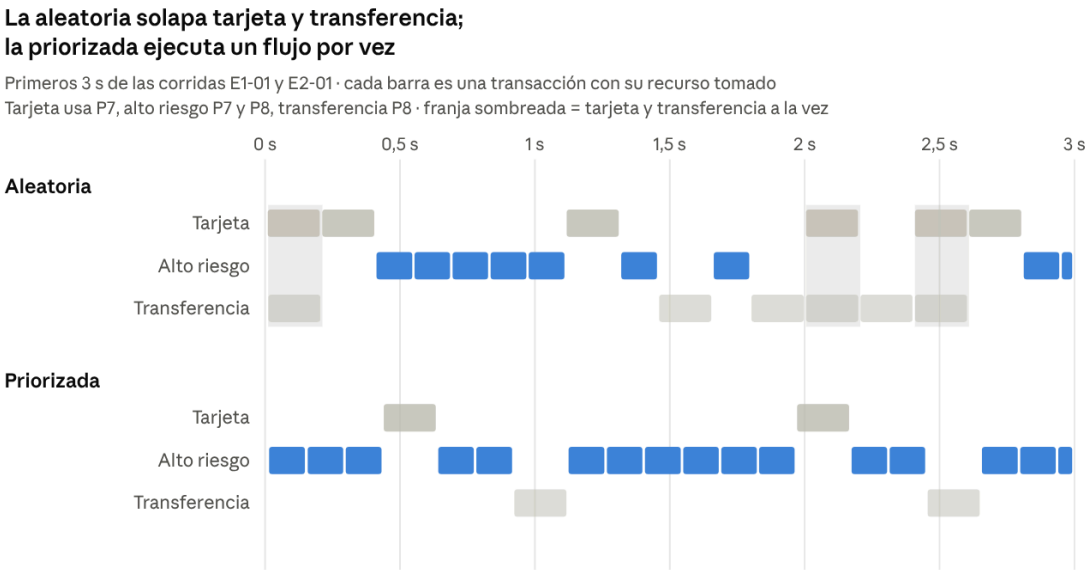


logs/resultados.csv · 50 corridas de 200 invariantes (E1 a E8)

- **Tiempos base:** aleatoria 29,40 s ± 0,75 (de 28,43 a 30,83 s) y priorizada 30,52 s ± 0,03. Las dos cumplen el requerimiento 10a. La priorizada casi no varía porque su secuencia de turnos es siempre la misma.
- **Modelo contra medición:** aplicando el modelo a la secuencia de ruteo real de cada corrida (sacada del log), la medición queda entre 0,08 y 0,20 s por encima en las 50 corridas. Esa diferencia es el costo fijo de arrancar los hilos, redondear las esperas y sincronizar: menos del 0,5 % del total.
- **Scoring al doble (T5 = 280 ms):** la priorizada sube 22,4 s (160 × 140 ms, exacto) y la aleatoria 9,5 s, porque alto riesgo pesa 160 veces en una y unas 66 en la otra. La priorizada pasa a estar fuera del rango de 20 a 40 s.

- **Tarjeta al doble (T2 = 240, T3 = 160 ms):** la aleatoria sube 11,8 s y la priorizada solo 4,0 s (20 × 200 ms). En la aleatoria sube menos que 66 × 200 ms = 13,2 s porque parte de esas tarjetas se solapa con transferencias.
- **Todo a la mitad:** 14,8 s y 15,3 s, la mitad del modelo base más el mismo costo fijo de unos 0,1 s: la duración escala con los α.

12.3 Paralelismo



logs/aleatoria_base_E1-01_eventos.csv y logs/priorizada_base_E2-01_eventos.csv · primeros 3 s

- Según los logs de eventos, en la aleatoria tarjeta y transferencia trabajan a la vez 6,75 s por corrida en promedio; P7 está ocupado el 79 % del tiempo y P8 el 76 %.
- En la priorizada el solape es 0 s: alto riesgo toma los dos recursos y los otros flujos nunca quedan seguidos. Los recursos están ocupados el 87 % del tiempo, pero de a una transacción por vez.

12.4 Conclusiones del análisis temporal

- La duración la fijan las transiciones temporales; el monitor agrega un costo fijo de unos 0,1 s por corrida.
- Priorizar alto riesgo reduce el paralelismo: ese flujo bloquea los dos recursos, así que la priorizada no aprovecha que tarjeta y transferencia pueden ir en paralelo.
- El tiempo que conviene optimizar depende de la política: con la priorizada, cada milisegundo de T5 suma 0,16 s al total; con la aleatoria, unos 0,066 s.

13. Conclusiones

El monitor ejecuta la red sin conocerla y cumple los requerimientos del enunciado: con cada política completa 200 invariantes en unos 30 s, verifica los invariantes y termina sin hilos activos.

- **Monitor correcto.** Las 50 corridas del plan y otras 600 sin tiempo (perfil `sin_tiempo`, para forzar la contención) terminaron sin deadlocks, con todos los P-invariantes válidos, residuo vacío en la regex y ningún hilo vivo.
- **La política tiene que actuar en el conflicto.** Si solo elige a quién despertar, alto riesgo queda entre 11 % y 24 % aun priorizándolo. Con el turno del conflicto, la aleatoria reparte 1/3 por flujo y la priorizada exactamente 8:1:1.
- **El tiempo se explica con un modelo simple.** El modelo de despacho secuencial predice cada corrida con 0,11 s de diferencia promedio. Priorizar alto riesgo cuesta paralelismo: la priorizada nunca solapa tarjeta y transferencia.
- **Límites y decisiones.** Los pesos 1:8:1, $\beta = \infty$ y los α son decisiones nuestras; la prioridad estricta dejaría sin servicio a tarjeta y transferencia. Las capturas de PIPE de la sección 3 se agregan aparte.
- **Mejoras posibles.** Una prioridad dinámica (por ejemplo, con envejecimiento) podría priorizar alto riesgo sin fijar la proporción, y la configuración de la red podría leerse de un archivo en lugar de una clase.

14. Anexos

14.1 Cómo ejecutar

Requiere JDK 8 o superior y ninguna biblioteca externa. Desde la carpeta del proyecto (los mismos comandos sirven en Linux, macOS y Windows):

```
mkdir out
javac -encoding UTF-8 -d out -sourcepath src src/Main.java src/Experimentos.java
java -cp out Main                        # política aleatoria, tiempos base
java -cp out Main priorizada             # política priorizada
java -cp out Main aleatoria 02 base      # política, etiqueta de la corrida y perfil
de tiempos
java -cp out Experimentos                # plan completo E1 a E8
```

También están `correr.sh` y `correr.bat`. En un IDE alcanza con marcar `src/` como carpeta de fuentes y ejecutar `Main`.

14.2 Estructura del proyecto

```
src/
  Main.java, Experimentos.java
  simulacion/         Simulacion, Resultado
  monitor/            MonitorInterface, Monitor, Colas
  monitor/politicas/  Politica, PoliticaPorConflictos, PoliticaAleatoria,
PoliticaPriorizada
  red/                RedDePetri, ControlTemporal, ConfiguracionRed, Segmento
  hilos/              Disparador, FabricaHilos
  registro/           Log, AnalizadorInvariantes
docs/                 diagrama_clases.png, diagrama_secuencia.png, figuras de la red
herramientas/         analisis_red.py, comparacion/SoloDespertar.java
logs/                 logs de las 50 corridas del informe y resultados.csv
```

14.3 Extracto de un log de resumen (corrida E1-01)

```
Politica: aleatoria
Perfil de tiempos: base  alfa(ms) T2=120 T3=80 T5=140 T7=70 T8=130  beta = infinito
Duracion hasta el invariante 200: 29.536 s
Invariantes de transicion (expresion regular sobre el log):
  IT1 tarjeta          68  ( 34.0 %)
  IT2 alto riesgo      71  ( 35.5 %)
  IT3 transferencia    61  ( 30.5 %)
  Total               200
  Residuo de la regex: vacio (3 pasadas)
Disparos por transicion:
  T0=200  T1=68  T2=68  T3=68  T4=71  T5=71  T6=61  T7=61  T8=61  T9=200
Disparos por hilo:
  SA-1: 69  SA-2: 66  SA-3: 65  SB-1: 204  SC-1: 142  SD-1: 183  SE-1: 66  SE-2: 67
SE-3: 67
Verificaciones de P-invariantes (una por disparo): 929, todas OK
Marcado final: [3, 0, 0, 0, 0, 0, 0, 1, 1, 0] = M0
Hilos creados: 9 - hilos activos al terminar: 0
Resultado: CORRECTO
```

14.4 Las 50 corridas del plan de experimentos

Tiempos: base; scoring ×2 (T5 = 280 ms); tarjeta ×2 (T2 = 240, T3 = 160 ms); mitad (todos los α a la mitad). IT1 = tarjeta, IT2 = alto riesgo, IT3 = transferencia.

Corrida	Política	Tiempos	Duración (s)	IT1	IT2	IT3	Cheques P-inv	Residuo	Hilos activos
E1-01	aleatoria	base	29,54	68	71	61	929	vacío	0
E1-02	aleatoria	base	29,25	79	58	63	942	vacío	0
E1-03	aleatoria	base	28,44	73	61	66	939	vacío	0
E1-04	aleatoria	base	28,87	65	64	71	936	vacío	0
E1-05	aleatoria	base	29,47	76	64	60	936	vacío	0
E1-06	aleatoria	base	30,09	70	67	63	933	vacío	0
E1-07	aleatoria	base	28,43	65	71	64	929	vacío	0
E1-08	aleatoria	base	30,83	69	71	60	929	vacío	0
E1-09	aleatoria	base	29,93	62	66	72	934	vacío	0
E1-10	aleatoria	base	29,20	67	68	65	932	vacío	0
E2-01	priorizada	base	30,60	20	160	20	840	vacío	0
E2-02	priorizada	base	30,54	20	160	20	840	vacío	0
E2-03	priorizada	base	30,52	20	160	20	840	vacío	0

Corrida	Política	Tiempos	Duración (s)	IT1	IT2	IT3	Cheques P-inv	Residuo	Hilos activos
E2-04	priorizada	base	30,52	20	160	20	840	vacío	0
E2-05	priorizada	base	30,50	20	160	20	840	vacío	0
E2-06	priorizada	base	30,50	20	160	20	840	vacío	0
E2-07	priorizada	base	30,50	20	160	20	840	vacío	0
E2-08	priorizada	base	30,50	20	160	20	840	vacío	0
E2-09	priorizada	base	30,49	20	160	20	840	vacío	0
E2-10	priorizada	base	30,49	20	160	20	840	vacío	0
E3-01	aleatoria	scoring ×2	40,59	62	75	63	925	vacío	0
E3-02	aleatoria	scoring ×2	38,89	70	62	68	938	vacío	0
E3-03	aleatoria	scoring ×2	39,25	61	69	70	931	vacío	0
E3-04	aleatoria	scoring ×2	37,79	69	66	65	934	vacío	0
E3-05	aleatoria	scoring ×2	38,11	66	60	74	940	vacío	0
E4-01	priorizada	scoring ×2	52,90	20	160	20	840	vacío	0
E4-02	priorizada	scoring ×2	52,89	20	160	20	840	vacío	0
E4-03	priorizada	scoring ×2	52,90	20	160	20	840	vacío	0
E4-04	priorizada	scoring ×2	52,90	20	160	20	840	vacío	0
E4-05	priorizada	scoring ×2	52,88	20	160	20	840	vacío	0
E5-01	aleatoria	tarjeta ×2	38,85	57	82	61	918	vacío	0
E5-02	aleatoria	tarjeta ×2	40,02	66	75	59	925	vacío	0
E5-03	aleatoria	tarjeta ×2	42,07	71	64	65	936	vacío	0
E5-04	aleatoria	tarjeta ×2	44,23	74	58	68	942	vacío	0
E5-05	aleatoria	tarjeta ×2	40,93	62	73	65	927	vacío	0
E6-01	priorizada	tarjeta ×2	34,52	20	160	20	840	vacío	0
E6-02	priorizada	tarjeta ×2	34,49	20	160	20	840	vacío	0
E6-03	priorizada	tarjeta ×2	34,49	20	160	20	840	vacío	0
E6-04	priorizada	tarjeta ×2	34,50	20	160	20	840	vacío	0

Corrida	Política	Tiempos	Duración (s)	IT1	IT2	IT3	Cheques P-inv	Residuo	Hilos activos
E6-05	priorizada	tarjeta ×2	34,49	20	160	20	840	vacío	0
E7-01	aleatoria	mitad	15,17	61	61	78	939	vacío	0
E7-02	aleatoria	mitad	14,69	60	64	76	936	vacío	0
E7-03	aleatoria	mitad	14,56	72	64	64	936	vacío	0
E7-04	aleatoria	mitad	14,89	73	60	67	940	vacío	0
E7-05	aleatoria	mitad	14,66	63	64	73	936	vacío	0
E8-01	priorizada	mitad	15,29	20	160	20	840	vacío	0
E8-02	priorizada	mitad	15,30	20	160	20	840	vacío	0
E8-03	priorizada	mitad	15,30	20	160	20	840	vacío	0
E8-04	priorizada	mitad	15,31	20	160	20	840	vacío	0
E8-05	priorizada	mitad	15,30	20	160	20	840	vacío	0

14.5 Referencias

- Enunciado del Trabajo Práctico Final, Programación Concurrente y Paralela 2026, FCEFyN, UNC.
- Clases de la cátedra: Monitores; Thread Factory y Executors; Propiedades de las redes de Petri; Redes de Petri con tiempo; Expresiones regulares; diagramas de secuencia del monitor sin tiempo y con tiempo.
- L. Ventre y O. Micolini, Algoritmos para determinar cantidad y responsabilidad de hilos en sistemas embebidos modelados con Redes de Petri S3PR, CACIC 2021.